



GIT CHEATSHEET

by *CodeWithHarry*

1. SETUP & CONFIG

<code>git config --global user.name "Your Name"</code>	Set global username
<code>git config --global user.email "you@example.com"</code>	Set global email
<code>git config --global core.editor code</code>	Set default editor
<code>git config --list</code>	List all config
<code>git config --global --list</code>	List global config
<code>git config --local --list</code>	List local config

3. DAILY WORKFLOW

<code>git status</code>	Show working tree status
<code>git add <file></code>	Stage a specific file
<code>git add .</code>	Stage all changes
<code>git commit -m "message"</code>	Commit staged changes
<code>git commit -am "message"</code>	Stage all & commit
<code>git log</code>	Show commit history
<code>git log --oneline --graph --all</code>	Compact commit history graph
<code>git diff</code>	Show unstaged changes
<code>git diff --staged</code>	Show staged changes
<code>git show <commit></code>	Show details of a commit

5. REMOTES

<code>git remote -v</code>	List remotes
<code>git remote add origin <url></code>	Add a new remote
<code>git remote remove <name></code>	Remove a remote
<code>git fetch <remote></code>	Fetch changes (no merge)
<code>git pull</code>	Fetch + merge
<code>git pull --rebase</code>	Fetch + rebase
<code>git push <remote> <branch></code>	Push branch to remote
<code>git push -u origin <branch></code>	Push & set upstream
<code>git push --tags</code>	Push all tags
<code>git push --force</code>	Force push (use with caution)

7. UNDO & REVERT

<code>git checkout -- <file></code>	Discard changes in file
<code>git reset <file></code>	Unstage a file
<code>git reset --soft <commit></code>	Move HEAD (keep changes)
<code>git reset --mixed <commit></code>	Move HEAD (unstage changes)
<code>git reset --hard <commit></code>	Move HEAD (discard changes)
<code>git revert <commit></code>	Revert a commit (safe)
<code>git revert HEAD</code>	Revert the last commit
<code>git commit --amend</code>	Amend last commit
<code>git rebase <branch></code>	Rebase current branch
<code>git rebase -i HEAD~n</code>	Interactive rebase (last n commits)

2. START A PROJECT

<code>git init</code>	Initialize a new Git repository
<code>git clone <repo-url></code>	Clone (download) a repository
<code>git clone <repo-url> <dir></code>	Clone into a specific directory

4. BRANCHING & MERGING

<code>git branch</code>	List branches
<code>git branch -r</code>	List remote branches
<code>git branch -a</code>	List all branches
<code>git branch <branch-name></code>	Create a new branch
<code>git checkout <branch-name></code>	Switch to a branch
<code>git checkout -b <branch-name></code>	Create & switch to a branch
<code>git merge <branch-name></code>	Merge branch into current
<code>git branch -d <branch></code>	Delete a branch (safe)
<code>git branch -D <branch></code>	Force delete a branch

6. STASH

<code>git stash</code>	Stash local changes
<code>git stash push -m "message"</code>	Stash with a message
<code>git stash list</code>	List stashes
<code>git stash show</code>	Show stash changes
<code>git stash pop</code>	Apply & remove latest stash
<code>git stash apply</code>	Apply latest stash (keep it)
<code>git stash drop</code>	Remove latest stash
<code>git stash clear</code>	Remove all stashes

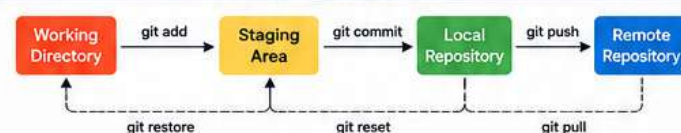
8. TAGGING

<code>git tag</code>	List tags
<code>git tag <tagname></code>	Create a lightweight tag
<code>git tag -a <tagname> -m "msg"</code>	Create an annotated tag
<code>git show <tagname></code>	Show tag details
<code>git push origin <tagname></code>	Push a tag
<code>git push origin --tags</code>	Push all tags

9. SUBMODULES

<code>git submodule add <url> <path></code>	Add a submodule
<code>git submodule init</code>	Initialize submodules
<code>git submodule update</code>	Update submodules
<code>git submodule update --init --recursive</code>	Init & update recursively
<code>git submodule foreach git pull origin main</code>	Pull in all submodules

COMMON WORKFLOW



OTHER USEFUL COMMANDS

<code>git help <command></code>	Get help for a command
<code>git version</code>	Show Git version
<code>git config --list</code>	Show all configuration
<code>git remote -v</code>	List remotes
<code>git log --graph --oneline --all</code>	View compact history

Happy Coding!